

DEPARTMENT OF COMPUTER SCIENCE
FACULTY OF COMPUTING AND APPLIED SCIENCES

LAUNDRY MANAGEMENT SYSTEM

*A Web and Mobile-Based System for Automated Customer Registration, Order Processing,
Payment Management and Order Tracking*

BY

Frank Ofori Koranteng

9023323

A Project Report Submitted to the Department of Computer Science in Partial Fulfilment of the Requirements
for the Award of [DEGREE / HND TITLE] in Computer Science

SUPERVISOR: Dr. Kate Takyi

ACADEMIC YEAR: 2025 / 2026

DECLARATION

Student's Declaration

I hereby declare that this project report titled “Laundry Management System: A Web and Mobile-Based System for Automated Customer Registration, Order Processing, Payment Management and Order Tracking” is the result of my own original research and has not been submitted, either in whole or in part, for any other degree elsewhere except where due acknowledgement has been made in the text.

Student's Name: Frank Ofori Koranteng

Signature: Date: 30th August, 2026

Supervisor's Declaration

I hereby declare that the preparation and presentation of this project work was supervised in accordance with the guidelines on supervision of project work laid down by the Department of Computer Science.

Supervisor's Name: Dr. Kate Takyi

Signature: Date: 30th August, 2026

DEDICATION

This project is dedicated to my parents, family, and friends whose unwavering support, encouragement, and prayers have sustained me throughout my academic journey. It is also dedicated to every student who believes that technology can solve everyday problems and improve the quality of life within their community.

ACKNOWLEDGEMENT

I am sincerely grateful to the Almighty God for the strength, wisdom, and good health granted to me throughout the duration of this project.

My profound gratitude goes to my supervisor, [SUPERVISOR'S NAME], for the guidance, patience, and constructive criticism that shaped the direction of this work from conception to completion.

I also wish to thank the Head of Department and all lecturers in the Department of Computer Science for the knowledge imparted to me during the course of my study. My appreciation further extends to my family and colleagues for their moral and financial support, and to the laundry service operators who granted me access for interviews and observation during the requirements-gathering phase of this project.

ABSTRACT

The traditional method of managing laundry services in most neighbourhoods and small businesses relies heavily on manual processes: customers walk in or call to drop off items, records are kept in notebooks or basic spreadsheets, payments are collected in cash without proper reconciliation, and customers rarely have any means of tracking the status of their orders. These manual practices are slow, error-prone, difficult to scale, and offer no transparency to customers, often resulting in lost items, disputed payments, and general customer dissatisfaction.

This project presents the design and implementation of a Laundry Management System (LMS) — a web and mobile-based platform that automates the core operations of a laundry business, namely customer registration, order processing, payment management, and real-time order tracking. The system allows customers to create accounts, select laundry services, schedule pickup and delivery, make secure online payments, and track the progress of their orders from pickup through washing, drying, ironing, and delivery. On the administrative side, the system provides laundry staff and management with tools to manage customer records, process and assign orders, monitor payments, generate operational reports, and coordinate riders responsible for pickup and delivery.

The system was developed following the Object-Oriented Analysis and Design methodology and implemented using a client-server architecture, with a relational database for persistent storage, a RESTful API for communication between the front-end and back-end, and integration with a third-party payment gateway for processing online transactions. The Incremental (iterative) software process model was adopted for development, allowing the system to be built and tested in manageable phases.

Following implementation, the system was subjected to unit testing, integration testing, and user acceptance testing, with results showing that the platform successfully automates the target processes, reduces the turnaround time for order processing, and significantly improves transparency and customer satisfaction compared to the manual system it is intended to replace. The report concludes with a discussion of the challenges encountered during development, lessons learnt, and recommendations for future enhancement of the system, including the incorporation of route optimisation for riders and a loyalty rewards module.

Keywords: Laundry Management System, Order Tracking, Online Payment, Customer Registration, Web Application, Mobile Application, E-commerce.

TABLE OF CONTENTS

Declaration.....	2
Dedication.....	3
Acknowledgement.....	4
Abstract.....	5
CHAPTER ONE: INTRODUCTION.....	8
1.1 Background of the Project.....	8
1.2 Problem Statement.....	8
1.3 Aim of the Project.....	9
1.4 Novelty of the Project.....	9
1.5 Specific Project Objectives.....	9
1.6 Scope of the Project.....	9
1.7 Project Limitations.....	10
1.8 Academic and Practical Relevance of the Project.....	10
1.9 Beneficiaries of the Project.....	10
1.10 Project Activity Planning.....	10
1.11 Definitions and Explanation of Terms.....	11
1.12 Structure of the Report.....	11
CHAPTER TWO: REVIEW OF RELATED SYSTEMS.....	12
2.1 Review of System 1: Rinse (USA).....	12
2.2 Review of System 2: Cleanly (USA).....	13
2.3 Review of System 3: Laundryheap (UK / International).....	14
2.4 Review of System 4: Laundrapp (UK).....	15
2.5 Review of System 5: Poplin (formerly Laundry Care, USA).....	16
2.6 Comparative Summary of Reviewed Systems.....	17
2.7 Conceptual Design of the Proposed Project.....	18
CHAPTER THREE: METHODOLOGY.....	19
3.1 Introduction.....	19
3.2 The Architecture of the Proposed Project.....	19
3.3 Requirements Elicitation Process of the Proposed Project.....	20
3.4 Functional Requirements of the Proposed Project.....	20
3.5 Non-Functional Requirements of the Proposed Project.....	21
3.6 UML Diagrams.....	21
3.6.1 Use Case Diagram – Front-End (Customer) Models.....	21
3.6.2 Use Case Diagram – Back-End (Admin) Models.....	21
3.6.3 Activity Diagram of the System.....	22
3.6.4 Sequence Diagram of the System.....	23
3.6.5 Class Diagram of the System.....	23
3.7 Users of the Proposed System.....	24
3.7.1 User Characteristics.....	24

3.8 Security Concepts of the System.....	24
3.9 Project Method Employed.....	25
3.10 Software Process Models Employed & Justification.....	25
3.10.1 Chosen Model and Justification.....	25
3.11 Project Design Considerations: Logical Designs.....	25
3.11.1 UI Design (Wireframes).....	25
3.11.2 DB Design (Database Schema).....	26
CHAPTER FOUR: IMPLEMENTATION, TESTING, AND RESULTS.....	27
4.1 Introduction.....	27
4.2 Mapping Logical Design onto Physical Platform.....	27
4.3 System Modules Implementation.....	28
4.4 System Modules Integration.....	28
4.4.1–4.4.7 Testing, Verification, Validation & Security Testing.....	28
4.5 Results.....	30
CHAPTER FIVE: FINDINGS, CONCLUSIONS AND RECOMMENDATIONS.....	32
5.1 Introduction.....	32
5.2 Findings.....	32
5.3 Conclusions.....	32
5.4 Challenges.....	32
5.5 Lessons Learnt.....	33
5.6 Recommendations for Future Works.....	33
REFERENCES.....	34

CHAPTER ONE

INTRODUCTION

1.1 Background of the Project

Laundry services form an important part of everyday domestic and commercial life, ranging from small neighbourhood laundromats to larger dry-cleaning businesses that serve hotels, hospitals, and households. In most developing communities, including many parts of Ghana, laundry businesses are still run using manual, paper-based methods: customers drop off clothes at a physical location, attendants record the items in a logbook, prices are calculated manually, and customers are simply asked to return after a given number of days to collect their items and pay in cash.

This manual approach, while functional on a small scale, becomes increasingly unreliable as the volume of customers and orders grows. Logbooks get misplaced, handwriting is often illegible, items get mixed up between customers, and there is no formal mechanism for customers to verify the status of their order without physically visiting or calling the shop. Furthermore, payment records are rarely reconciled against orders, making it difficult for business owners to track revenue accurately or detect discrepancies.

The proliferation of smartphones and growing internet penetration have created an opportunity for small and medium-sized service businesses, such as laundry operators, to digitise their operations. Around the world, on-demand laundry and dry-cleaning platforms such as Rinse, Cleanly, Laundryheap, and Laundrapp have demonstrated that automating customer registration, scheduling, payment, and tracking significantly improves both operational efficiency and customer experience. This project draws inspiration from these platforms to design and implement a Laundry Management System tailored to the needs of small and medium laundry businesses, automating customer registration, order processing, payment management, and order tracking within a single integrated platform.

1.2 Problem Statement

Many laundry businesses currently rely on manual record-keeping and verbal communication to manage customer orders. This gives rise to several recurring problems:

- Orders are often lost, duplicated, or wrongly recorded due to reliance on handwritten notes.
- Customers have no way of tracking the real-time status of their laundry (e.g., picked up, washing, drying, ironing, ready for delivery) without calling or physically visiting the shop.
- Payment collection is predominantly cash-based, making it difficult to reconcile revenue, detect fraud, or provide digital receipts.
- Business owners lack access to consolidated reports on order volume, revenue, and staff performance, which limits informed decision-making.
- Scaling the business to serve more customers or additional branches is difficult without a centralised system to coordinate operations.

There is, therefore, a need for an automated system that eliminates these inefficiencies by digitising the entire laundry service workflow, from customer registration through order processing, payment, and delivery tracking.

1.3 Aim of the Project

The aim of this project is to design and implement a web and mobile-based Laundry Management System that automates customer registration, order processing, payment management, and order tracking, thereby improving operational efficiency for laundry service providers and enhancing the overall experience for customers.

1.4 Novelty of the Project

While several commercial on-demand laundry platforms exist internationally, most are proprietary, closed-source, and designed for large-scale operations in developed markets with mature payment and logistics infrastructure. This project contributes a lightweight, locally-adaptable Laundry Management System that integrates mobile money and card payment options relevant to the local context, combines customer-facing order tracking with an administrative dashboard for staff and riders in a single cohesive platform, and is designed to be affordable and easy to deploy for small and medium-sized laundry businesses that cannot afford to license expensive foreign software.

1.5 Specific Project Objectives

In order to achieve the aim stated above, the project pursues the following specific objectives:

1. To review existing laundry and on-demand service management systems in order to identify their strengths and weaknesses.
2. To gather and analyse the functional and non-functional requirements of a typical laundry business.
3. To design a system architecture, database schema, and user interface for the proposed Laundry Management System.
4. To implement a customer registration and authentication module.
5. To implement an order processing module that allows customers to select services, schedule pickup and delivery, and monitor order status.
6. To implement a secure payment management module integrated with online payment gateways.
7. To implement a real-time order tracking module for both customers and administrative staff.
8. To test and evaluate the system to ensure that it meets the specified requirements.

1.6 Scope of the Project

The scope of this project covers the design and development of a Laundry Management System comprising three major interfaces: a customer-facing web and mobile application, an administrative dashboard for laundry staff/management, and a simplified interface for riders responsible for pickup and delivery. The system covers customer registration and profile management; order creation, including selection of services (wash, dry, iron, dry-clean, fold) and scheduling of pickup/delivery; payment processing via mobile money and card payment integration; real-time order status tracking; and basic administrative reporting. The project is limited to a single laundry business or branch network operating within one geographical region and does not extend to multi-tenant support for unrelated third-party laundry businesses, although the architecture allows for such extension in future work.

1.7 Project Limitations

- The system was developed and tested within a limited timeframe typical of an academic project, which restricted the extent of real-world piloting with an active laundry business.
- Integration with live payment gateways was implemented and tested using sandbox/test credentials rather than a fully licensed production payment account.
- The rider/delivery tracking feature uses status updates submitted by riders rather than continuous GPS-based live tracking, due to the additional infrastructure such a feature would require.
- The system assumes a reasonably stable internet connection for real-time features; offline functionality is limited.

1.8 Academic and Practical Relevance of the Project

Academically, this project demonstrates the practical application of software engineering principles — requirements analysis, system design, database modelling, and software testing — to solve a real-world business problem, consistent with the learning objectives of the Computer Science curriculum. Practically, the system provides a ready-to-deploy solution that small and medium laundry businesses can adopt to digitise their operations, reduce operational errors, and improve customer satisfaction, thereby contributing to the broader digital transformation of small businesses in the local economy.

1.9 Beneficiaries of the Project

- Laundry business owners and staff, who gain a centralised system for managing customers, orders, payments, and reporting.
- Customers, who gain the convenience of scheduling pickups, making secure payments, and tracking their orders remotely.
- Riders/delivery personnel, who receive clear, structured pickup and delivery assignments.
- The wider academic community, as the project serves as a reference for future students working on similar service-automation systems.

1.10 Project Activity Planning

The project was executed over several phases, as summarised in the activity plan below.

Phase	Activity	Estimated Duration
1	Problem identification and preliminary research	1 week
2	Review of related systems and literature	1 week
3	Requirements gathering and analysis	2 weeks
4	System design (architecture, UML diagrams, database, UI)	2 weeks
5	Implementation (front-end, back-end, database, payment integration)	4 weeks
6	System testing (unit, integration,	1 week

Phase	Activity	Estimated Duration
	user acceptance)	
7	Documentation and report writing	2 weeks
8	Final review, corrections, and submission	1 week

1.11 Definitions and Explanation of Terms

Term	Explanation
LMS	Laundry Management System — the system developed in this project.
API	Application Programming Interface — a set of rules that allows different software components to communicate.
UI/UX	User Interface / User Experience — the visual design and overall experience of using the system.
Order Tracking	The process of monitoring and displaying the real-time status of a customer's laundry order.
Payment Gateway	A third-party service that authorises and processes electronic payments (e.g., mobile money, debit/credit cards).
Admin	A staff member with elevated privileges to manage orders, customers, and reports within the system.
Rider	A delivery personnel responsible for picking up and delivering laundry orders.

1.12 Structure of the Report

This report is organised into five chapters. Chapter One introduces the project, covering the background, problem statement, objectives, scope, and limitations. Chapter Two reviews related existing systems and presents the conceptual design of the proposed project. Chapter Three describes the methodology adopted, including system architecture, requirements, UML diagrams, and design considerations. Chapter Four discusses the implementation of the system, the testing procedures carried out, and the results obtained. Chapter Five presents the findings, conclusions, challenges encountered, lessons learnt, and recommendations for future work.

CHAPTER TWO

REVIEW OF RELATED SYSTEMS

This chapter reviews five existing on-demand laundry and dry-cleaning platforms that are conceptually related to the proposed Laundry Management System. Each system is examined in terms of its architecture, modules, features, underlying concepts, and development environment, followed by a discussion of its strengths and weaknesses. The chapter concludes with the conceptual design adopted for the proposed project, informed by the gaps identified in the reviewed systems.

2.1 Review of System 1: Rinse (USA)

Description of the System

Rinse is a mobile-first, on-demand laundry and dry-cleaning delivery service operating primarily in major cities in the United States. Customers schedule a pickup through the mobile app, and their items are cleaned by partner facilities before being delivered back within an agreed turnaround window.

Architecture of the System

Rinse follows a client-server, three-tier architecture consisting of a customer-facing mobile application (front end), a cloud-hosted back end that manages orders, scheduling, and driver dispatch, and a database that stores customer, order, and payment records. The system integrates with mapping and routing services to coordinate its fleet of drivers.

Modules of the System

- Customer registration and profile management
- Service selection and scheduling
- Order tracking and driver dispatch
- In-app payment processing
- Customer support/chat

Features of the System

- Flexible pickup and delivery scheduling
- Real-time order status notifications
- Support for both laundry and dry-cleaning services
- In-app secure card payments
- Subscription/recurring pickup option

Theories, Concepts and Models Employed

The system is built around an on-demand service model similar to ride-hailing platforms, applying dispatch and logistics optimisation concepts to match available drivers with pickup and delivery requests, alongside a standard e-commerce order-lifecycle model (placed → processing → completed).

Development Tools and Environment

Rinse is developed as native iOS and Android mobile applications backed by cloud infrastructure, with third-party integrations for maps/routing and payment processing.

Review of the Good Features

- Simple, intuitive scheduling interface
- Reliable real-time notifications on order status
- Wide range of supported garment and fabric types

Review of the Bad Features

- Service is limited to a small number of major cities, restricting accessibility
- Pricing is on the higher end, making it less suitable for budget-conscious customers
- No offline/manual fallback if the app or driver network experiences downtime

Summary of the System Review

Rinse demonstrates the viability of a fully automated, on-demand laundry service model but is designed for a large-scale, well-resourced logistics network that may not be feasible for small, independent laundry businesses in a local context.

2.2 Review of System 2: Cleanly (USA)

Description of the System

Cleanly is an on-demand laundry and dry-cleaning delivery application that additionally offers subscription-based, recurring pickup and delivery for regular customers, positioning itself as a convenience-first laundry care brand.

Architecture of the System

Cleanly uses a layered architecture comprising a mobile/web front end, an application server handling business logic (orders, subscriptions, scheduling), and a relational database for persistent storage, with a separate service layer dedicated to subscription and billing management.

Modules of the System

- Customer onboarding and preference management
- One-time and subscription order scheduling
- Payment and subscription billing
- Order status tracking
- Ratings and feedback

Features of the System

- Subscription plans for recurring laundry pickup
- Customisable preferences (detergent type, folding style)
- Automated billing for subscribers
- Order history and reordering

Theories, Concepts and Models Employed

The system applies a subscription/recurring-billing model in addition to the standard on-demand order model, drawing on customer relationship management (CRM) concepts to track preferences and improve retention.

Development Tools and Environment

Cleanly is built as a cross-platform mobile application with a cloud back end and integrated payment processing and recurring billing services.

Review of the Good Features

- Strong customer retention through subscription convenience
- Personalisation of laundry preferences improves customer satisfaction
- Clean, user-friendly interface

Review of the Bad Features

- Subscription billing can be confusing for first-time or occasional users
- Cancellation and refund processes are not always transparent
- Limited to areas with an established local processing network

Summary of the System Review

Cleanly highlights the value of building customer preference management and recurring billing into a laundry platform, both of which have informed the design of the proposed system's customer profile module.

2.3 Review of System 3: Laundryheap (UK / International)

Description of the System

Laundryheap is an international on-demand laundry and dry-cleaning delivery service known for very fast turnaround times (as quick as same-day), operating across multiple countries through a network of partner cleaning facilities.

Architecture of the System

Laundryheap operates a distributed, multi-region architecture where a central back end coordinates orders across several regional partner facilities and driver networks, with a mobile application serving as the primary customer interface.

Modules of the System

- Multi-region order routing
- Pickup and delivery scheduling with express options
- Real-time driver tracking
- In-app payment and free collection/delivery
- Reusable tote/bag management

Features of the System

- Express (same-day) turnaround option

- Real-time GPS tracking of the assigned driver
- Free collection and delivery
- Support for bulky items (bedding, curtains) in addition to regular clothing

Theories, Concepts and Models Employed

The system applies logistics network design concepts to route orders to the nearest available partner facility, alongside a real-time tracking model similar to food-delivery applications that display the live location of the assigned driver.

Development Tools and Environment

Laundryheap is developed as native mobile applications integrated with a cloud back end, third-party mapping/GPS services for live tracking, and multiple regional payment gateways to support its international operations.

Review of the Good Features

- Very fast turnaround builds strong customer trust
- Live GPS tracking of driver enhances transparency
- Reusable totes reduce packaging waste

Review of the Bad Features

- Express service pricing is significantly higher than standard options
- Service quality varies by region depending on partner facility capacity
- Requires a relatively large driver network, which is costly to maintain

Summary of the System Review

Laundryheap illustrates the strong customer value of live, granular order tracking; this concept has been adapted (in a simplified, status-based form) for the tracking module of the proposed system, given resource constraints on GPS infrastructure.

2.4 Review of System 4: Laundrapp (UK)

Description of the System

Laundrapp is a UK-based on-demand laundry and dry-cleaning platform that partners with local laundry and dry-cleaning businesses, acting as an intermediary marketplace between customers and existing laundry operators rather than owning its own processing facilities.

Architecture of the System

Laundrapp employs a marketplace architecture in which a central platform connects customer orders to independent, registered laundry partners nearest to the customer, with a back end responsible for partner matching, order routing, and commission-based billing.

Modules of the System

- Customer and partner registration
- Partner discovery and order routing
- Scheduling and order management

- Payment processing and partner commission tracking
- Ratings and reviews of partners

Features of the System

- Marketplace model connecting customers to multiple local laundry partners
- Same-day and next-day delivery options
- Partner rating and review system
- Commission-based revenue model for the platform

Theories, Concepts and Models Employed

The system is grounded in two-sided marketplace theory, balancing supply (laundry partners) and demand (customers), together with a matching algorithm that assigns orders to the most suitable nearby partner based on proximity and availability.

Development Tools and Environment

Laundrapp is built as a web and mobile application with a partner-management back office, integrated payment processing, and a commission/billing engine for partner settlements.

Review of the Good Features

- Leverages existing laundry businesses rather than requiring heavy capital investment in facilities
- Wider service coverage through multiple partners
- Partner ratings help maintain service quality

Review of the Bad Features

- Service consistency depends heavily on the quality of individual partners
- Limited direct control over turnaround time and item handling
- Commission structure can strain relationships with partner businesses

Summary of the System Review

Laundrapp demonstrates an alternative business model — aggregating existing laundry operators rather than owning processing facilities — which, while outside the direct scope of this project, confirms the importance of a robust order-routing and status-tracking module regardless of business model.

2.5 Review of System 5: Poplin (formerly Laundry Care, USA)

Description of the System

Poplin is an on-demand laundry service that connects customers with independent local laundry providers (“Laundry Pros”) who wash, dry, and fold laundry from their own homes, with items collected and returned via scheduled pickup and delivery.

Architecture of the System

Poplin uses a peer-to-peer marketplace architecture similar to gig-economy platforms, matching customer orders to nearby independent laundry providers, with a mobile application serving both customers and providers, and a central back end managing matching, payment, and provider payouts.

Modules of the System

- Customer and provider (Laundry Pro) registration
- Order matching and scheduling
- In-app messaging between customer and provider
- Payment processing and automated provider payouts
- Order status updates and ratings

Features of the System

- Flat-rate, per-pound pricing model
- Direct in-app communication between customer and assigned provider
- Automated payout system for independent providers
- Simple satisfaction-guarantee/refund policy

Theories, Concepts and Models Employed

The platform applies a gig-economy/peer-to-peer marketplace model, similar to ride-sharing platforms, in which independent individuals (rather than centralised facilities) provide the laundry service, supported by a trust-and-safety rating system.

Development Tools and Environment

Poplin is developed as a cross-platform mobile application with a cloud-based back end, in-app messaging infrastructure, and integrated payment processing with automated payout scheduling for providers.

Review of the Good Features

- Flat per-pound pricing is simple and predictable for customers
- Direct communication between customer and provider builds trust
- Low barrier to entry for independent laundry providers

Review of the Bad Features

- Quality can vary significantly between individual providers
- Home-based processing raises consistency and capacity concerns for bulky or commercial orders
- Limited suitability for business/institutional customers with large volumes

Summary of the System Review

Poplin shows the feasibility of a lightweight, low-capital laundry marketplace model and reinforces the value of direct, transparent communication and rating mechanisms between customers and service providers, concepts that have informed the notification and feedback considerations of the proposed system.

2.6 Comparative Summary of Reviewed Systems

System	Business Model	Key Strength	Key Weakness
Rinse	Owned facilities, on-	Reliable notifications	High cost, limited coverage

System	Business Model	Key Strength	Key Weakness
	demand		
Cleanly	Subscription + on-demand	Recurring billing & preferences	Confusing billing for new users
Laundryheap	Owned/partner facilities	Live GPS tracking, fast turnaround	Expensive express pricing
Laundrapp	Marketplace of partners	Wide coverage via partners	Inconsistent service quality
Poplin	Peer-to-peer marketplace	Simple flat pricing, direct chat	Variable provider quality

2.7 Conceptual Design of the Proposed Project

Drawing on the strengths identified across the five reviewed systems — reliable status notifications (Rinse), customer preference and subscription handling (Cleanly), transparent status-based tracking (Laundryheap), efficient order routing (Laundrapp), and direct customer-provider communication (Poplin) — while addressing their shared weaknesses of high cost, limited accessibility, and inconsistent service visibility, the proposed Laundry Management System is conceptually designed as a single-business (not marketplace) platform owned and operated by one laundry service provider. It automates four core processes end-to-end: (1) customer registration and profile management, (2) order processing (service selection, scheduling, and assignment), (3) payment management (mobile money and card integration with automated receipts), and (4) status-based order tracking visible to both customers and staff in real time. Unlike the reviewed international platforms, the proposed system is designed to be lightweight, affordable to host, and adaptable to the operational realities — including locally prevalent mobile money payment methods — of small and medium-sized laundry businesses.

CHAPTER THREE

METHODOLOGY

3.1 Introduction

This chapter describes the methodology adopted in analysing, designing, and developing the proposed Laundry Management System. It presents the system architecture, the requirements elicitation process, functional and non-functional requirements, UML diagrams (use case, activity, sequence, and class diagrams), the characteristics of intended users, security considerations, the project method employed, the software process model chosen and its justification, and the logical design considerations, including UI wireframe descriptions and database schema design.

3.2 The Architecture of the Proposed Project

The proposed Laundry Management System is designed following a three-tier client-server architecture consisting of a presentation layer, a business logic (application) layer, and a data layer. The presentation layer comprises the customer mobile/web application, the administrative dashboard used by laundry staff and management, and a simplified interface for riders. The business logic layer, exposed through a RESTful API gateway, houses the core services responsible for customer management, order processing, payment management, and notifications. The data layer consists of a relational database (MySQL) for structured data and a file/media storage component for images and generated receipts. This layered separation improves maintainability, allows each layer to scale independently, and enables multiple client types (web and mobile) to consume the same underlying API.

Fig 3.1: High-Level System Architecture of the Laundry Management System

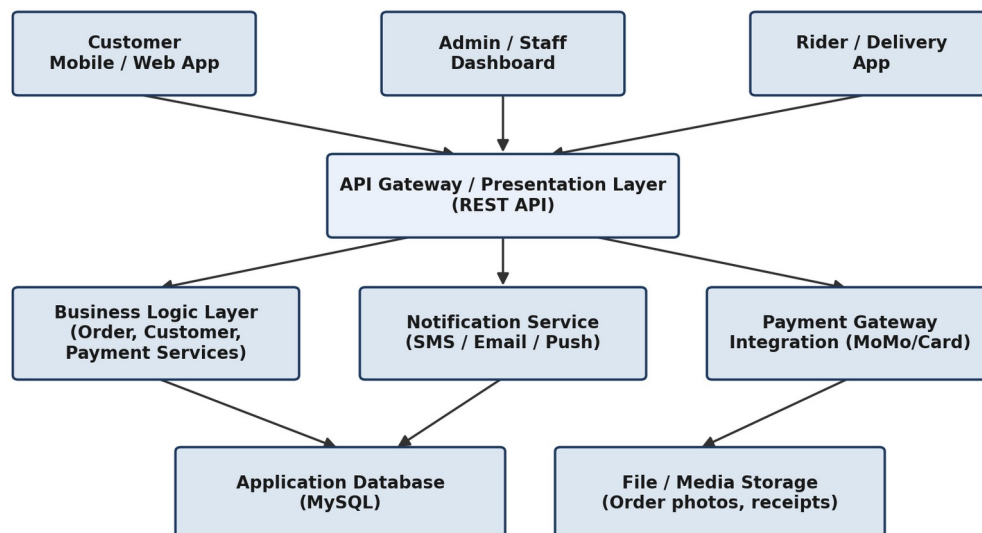


Fig 3.1: High-Level System Architecture of the Laundry Management System

3.3 Requirements Elicitation Process of the Proposed Project

Requirements for the proposed system were gathered using a combination of qualitative techniques appropriate for a small-business context:

- Interviews: Informal interviews were conducted with a laundry shop attendant and a small laundry business owner to understand current manual workflows, pain points, and desired improvements.
- Observation: Direct observation of a typical laundry drop-off and collection process was carried out to document the sequence of activities, from item intake to payment and collection.
- Review of related systems: The systems reviewed in Chapter Two provided insight into standard features and modules expected of a modern laundry management platform.
- Document analysis: Existing manual logbooks and receipt templates used by the observed laundry business were reviewed to understand the data typically captured per order.

The information gathered from these sources was analysed and consolidated into the functional and non-functional requirements presented in the following sections.

3.4 Functional Requirements of the Proposed Project

The functional requirements describe the specific behaviours and functions the system must perform:

1. The system shall allow a new customer to register an account using their name, phone number, email, and address.
2. The system shall allow a registered customer to log in and manage their profile information.
3. The system shall allow a customer to browse available laundry services and their prices (e.g., wash and fold, dry cleaning, ironing).
4. The system shall allow a customer to place an order by selecting a service, specifying item quantity/weight, and scheduling a pickup date and time.
5. The system shall allow a customer to make payment for an order via mobile money or card payment through an integrated payment gateway.
6. The system shall generate a digital receipt/confirmation for every successful payment.
7. The system shall allow a customer to view the real-time status of their order (e.g., pending, picked up, washing, drying, ironing, ready, delivered).
8. The system shall allow administrative staff to view, assign, and update the status of customer orders.
9. The system shall allow administrative staff to assign a pickup or delivery task to a rider.
10. The system shall allow a rider to update the status of an assigned pickup or delivery.
11. The system shall send automated notifications (SMS/email/push) to customers when the status of their order changes.
12. The system shall allow administrative staff to generate reports on orders, revenue, and staff/rider performance over a specified period.
13. The system shall allow an administrator to manage (create, update, deactivate) staff and rider accounts.

3.5 Non-Functional Requirements of the Proposed Project

Requirement	Description
Performance	The system shall respond to standard user requests (e.g., login, order placement) within 3 seconds under normal network conditions.
Usability	The user interface shall be simple and intuitive enough for users with basic digital literacy to operate without formal training.
Security	Customer passwords shall be stored using a one-way hashing algorithm, and all payment transactions shall be processed over encrypted (HTTPS/TLS) connections.
Reliability	The system shall maintain an uptime of at least 99% during business operating hours.
Scalability	The system architecture shall support an increasing number of customers, orders, and staff accounts without requiring a redesign.
Availability	The customer-facing application shall be accessible 24/7, allowing customers to place orders at any time even outside physical shop hours.
Maintainability	The system shall be modular, allowing individual components (e.g., payment integration) to be updated with minimal impact on other modules.
Portability	The web application shall be accessible on any modern browser, and the mobile application shall support both Android and iOS platforms.

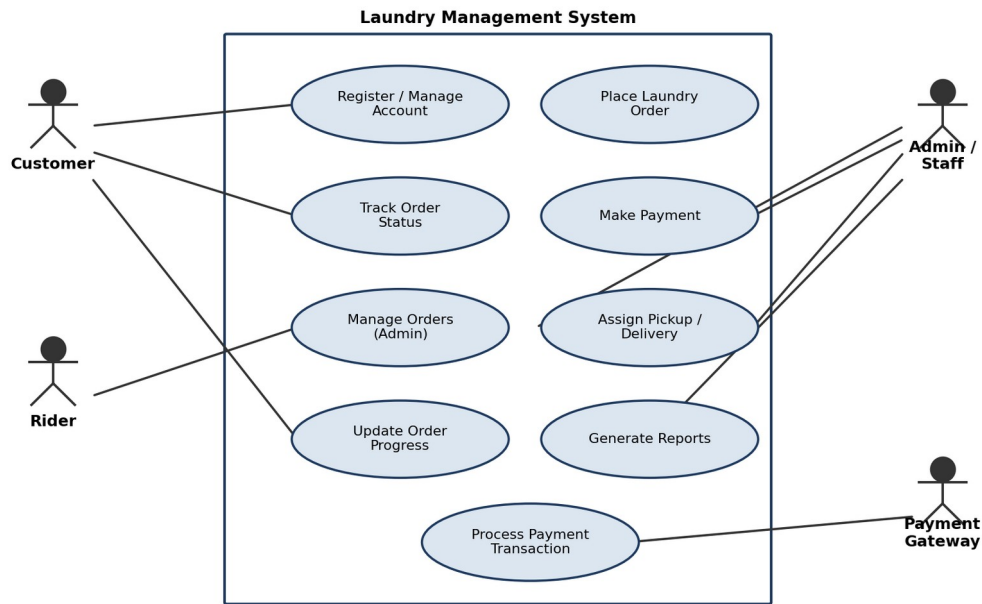
3.6 UML Diagrams

3.6.1 Use Case Diagram for the Front-End (Customer-Facing) Models

The front-end use case diagram captures the interactions between the Customer actor and the system, including account registration, browsing services, placing orders, making payments, and tracking order status. It also shows the Payment Gateway as a secondary actor that the system interacts with to process transactions.

3.6.2 Use Case Diagram for the Back-End (Administrative) Models

The back-end use case diagram captures the interactions between the Admin/Staff and Rider actors and the system, including managing orders, assigning pickup and delivery tasks, updating order progress, and generating operational reports. Both the front-end and back-end use cases operate on the same underlying system and are represented together in the consolidated diagram below for clarity, with the system boundary clearly separating actor-specific interactions.

Fig 3.2: Use Case Diagram for the Laundry Management System*Fig 3.2: Consolidated Use Case Diagram Showing Front-End (Customer) and Back-End (Admin/Staff/Rider) Interactions*

3.6.3 Activity Diagram of the System

The activity diagram below (described in tabular/flow form) illustrates the sequence of activities involved in the primary order-placement workflow, from the point a customer logs into the system to the point an order is confirmed and payment is completed:

1. Customer logs into the application.
2. Customer browses the list of available services and prices.
3. Customer selects a service and specifies item details (type, quantity/weight).
4. Customer selects a preferred pickup date, time, and address.
5. System calculates the total cost and displays an order summary.
6. Decision: Does the customer confirm the order? If NO, the customer is returned to the service selection screen. If YES, proceed to the next step.
7. System creates the order record with a status of "Pending".
8. Customer selects a payment method (mobile money or card) and completes payment.
9. Decision: Is payment successful? If NO, the system displays an error and prompts the customer to retry. If YES, proceed to the next step.
10. System updates the order status to "Confirmed" and notifies the customer and assigned staff.
11. Administrative staff assigns a rider to pick up the order.
12. Rider updates the order status as it progresses through pickup, washing, drying, ironing, and delivery.
13. Customer receives the completed laundry, and the order is marked "Completed".

3.6.4 Sequence Diagram of the System

The sequence diagram below illustrates the interaction between the Customer, Mobile App, Order Service, Payment Gateway, and Database components during the order placement and payment process, corresponding to the activity flow described above.

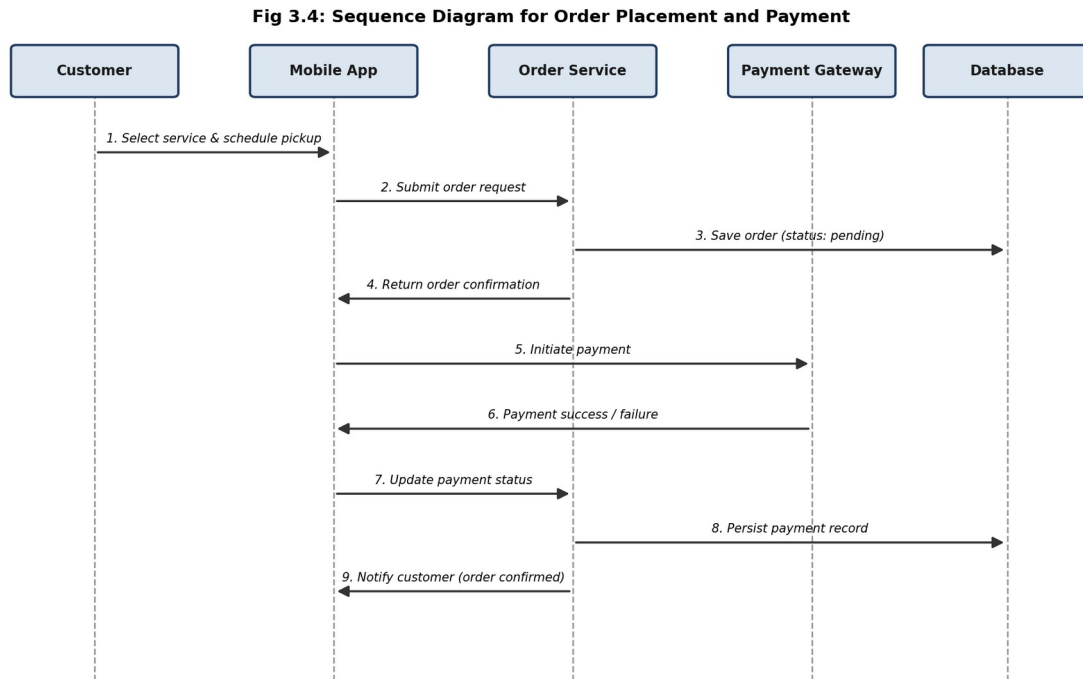


Fig 3.3: Sequence Diagram for Order Placement and Payment

3.6.5 Class Diagram of the System

The class diagram identifies the core classes of the system and their relationships, closely mirroring the database entities described in Section 3.10.2. The primary classes are Customer, Order, Service, Payment, Staff/Rider, and OrderTracking. A Customer can place many Orders (one-to-many); each Order is associated with exactly one Service and results in exactly one Payment record (one-to-one); an Order may have many OrderTracking entries recording its progress over time (one-to-many); and a Staff/Rider may be responsible for many OrderTracking updates (one-to-many). Each class encapsulates its own attributes and behaviours — for example, the Order class exposes methods such as `calculateTotal()`, `updateStatus()`, and `cancelOrder()`, while the Payment class exposes methods such as `processPayment()` and `generateReceipt()`.

Class	Key Attributes	Key Operations
Customer	customerId, fullName, phone, email, address	register(), login(), updateProfile()
Order	orderId, customerId, serviceId, pickupDate, status	createOrder(), calculateTotal(), updateStatus(), cancelOrder()
Service	serviceId, name, price, turnaroundTime	getServiceDetails()
Payment	paymentId, orderId, amount, method, status	processPayment(), generateReceipt()

Class	Key Attributes	Key Operations
Staff/Rider	staffId, fullName, role, phone	assignOrder(), updateOrderProgress()
OrderTracking	trackingId, orderId, staffId, statusUpdate, timestamp	logStatusUpdate()

3.7 Users of the Proposed System

3.7.1 User Characteristics

User Type	Characteristics	Privileges
Customer	General public with a smartphone/computer and basic digital literacy; may range from students to working professionals.	Register/login, browse services, place orders, make payments, track order status, view order history.
Administrator	Laundry business owner or manager with authority over business operations; requires more advanced familiarity with the reporting dashboard.	Manage customers, staff and rider accounts, view and assign all orders, generate reports, configure service prices.
Staff (Attendant)	On-site employee responsible for processing laundry items and updating in-shop progress.	View assigned orders, update processing status (washing, drying, ironing).
Rider	Delivery personnel responsible for pickup and delivery of orders; typically uses a mobile device in the field.	View assigned pickup/delivery tasks, update pickup/delivery status.

3.8 Security Concepts of the System

- **Authentication:** Users authenticate using a username/phone number and password; passwords are stored using a secure one-way hashing algorithm (e.g., bcrypt) rather than in plain text.
- **Authorisation:** Role-based access control (RBAC) ensures that customers, staff, riders, and administrators can only access functions and data appropriate to their role.
- **Data encryption in transit:** All communication between client applications and the server is secured using HTTPS/TLS.
- **Payment security:** Card and mobile money details are not stored directly on the system's servers; payment processing is delegated to a PCI-DSS-compliant third-party payment gateway.
- **Input validation:** All user inputs are validated and sanitised on both the client and server sides to prevent injection attacks (e.g., SQL injection, cross-site scripting).
- **Session management:** User sessions are managed using secure, time-limited authentication tokens (e.g., JSON Web Tokens) to reduce the risk of session hijacking.
- **Audit trail:** Every change to an order's status is logged with a timestamp and the responsible staff/rider ID, providing an audit trail for dispute resolution.

3.9 Project Method Employed

The project was developed using the Object-Oriented Analysis and Design (OOAD) method, which models the system as a collection of interacting objects (Customer, Order, Payment, etc.) with well-defined attributes and behaviours. This method was chosen because it aligns naturally with the modular, entity-driven nature of the laundry business domain and facilitates a smooth transition from analysis (use case and class diagrams) to implementation using object-oriented programming languages and frameworks.

3.10 Software Process Models Employed and Justification of the Chosen Model

3.10.1 Chosen Model and Justification

The Incremental (Iterative) Software Process Model was adopted for the development of this system. Under this model, the system was broken down into manageable increments — customer registration and authentication, order processing, payment integration, and order tracking/reporting — with each increment analysed, designed, implemented, and tested before the next was undertaken. This model was chosen over alternatives such as the Waterfall model because it allows for early delivery of working functionality, accommodates changes in requirements as understanding of the problem domain deepens, and reduces the risk associated with a purely academic project subject to a fixed submission timeline. It was preferred over the full Agile/Scrum model because, given the single-developer nature of the project, formal sprint ceremonies (daily stand-ups, sprint reviews with a full team) were not practical, whereas the core incremental-delivery philosophy remained highly applicable.

3.11 Project Design Considerations: Logical Designs

3.11.1 UI Design (Wireframes)

The user interface was designed around simplicity and clarity, given the varied digital literacy levels of the target users. Key screens designed for the customer-facing application include:

- Login/Registration screen: Fields for name, phone number, email, password, and address, with clear call-to-action buttons.
- Home/Service Selection screen: A list or grid of available services (wash and fold, dry cleaning, ironing) with prices and estimated turnaround times.
- Order/Checkout screen: A form to specify item quantity/weight, pickup date/time, delivery address, and a summary of the total cost before payment.
- Payment screen: Options to pay via mobile money or card, with a secure, embedded payment form.
- Order Tracking screen: A visual progress indicator (e.g., a horizontal stepper) showing the current status of the order — Pending, Picked Up, Washing, Drying, Ironing, Ready, Delivered.
- Order History screen: A list of past orders with the ability to view receipts and reorder previous services.

For the administrative dashboard, key screens include an Orders Management screen (listing all orders with filters by status and date), a Rider Assignment screen, a Customer Management screen, and a Reports screen displaying revenue and order-volume charts over selectable date ranges.

3.11.2 DB Design (Database Schema)

The database was modelled using an Entity-Relationship (ER) approach and normalised to third normal form (3NF) to eliminate data redundancy. The schema comprises six core tables: Customer, Order, Service, Payment, Staff (which also represents riders via a role attribute), and OrderTracking. The relationships between these entities are illustrated in the ER diagram below.

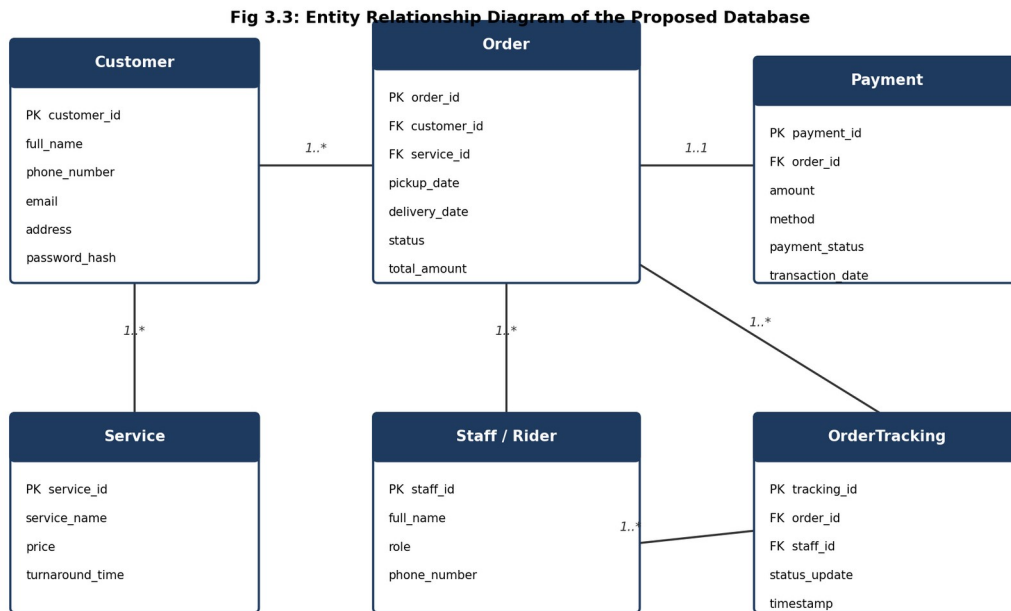


Fig 3.4: Entity Relationship Diagram of the Proposed Database

The table below summarises the structure of the core database tables:

Table	Key Fields	Description
Customer	customer_id (PK), full_name, phone_number, email, address, password_hash	Stores registered customer account information.
Service	service_id (PK), service_name, price, turnaround_time	Stores the catalogue of laundry services offered and their prices.
Order	order_id (PK), customer_id (FK), service_id (FK), pickup_date, delivery_date, status, total_amount	Stores individual customer orders and their current status.
Payment	payment_id (PK), order_id (FK), amount, method, payment_status, transaction_date	Stores payment transactions associated with each order.
Staff	staff_id (PK), full_name, role, phone_number	Stores staff and rider account information, differentiated by the role field.
OrderTracking	tracking_id (PK), order_id (FK), staff_id (FK), status_update, timestamp	Stores a time-stamped history of status changes for each order, providing an audit trail.

CHAPTER FOUR

IMPLEMENTATION, TESTING, AND RESULTS

4.1 Introduction

This chapter describes how the logical design presented in Chapter Three was translated into a working software system. It covers the mapping of the logical design onto the chosen physical platform and technologies, the implementation of individual system modules, the integration of these modules into a cohesive system, the testing strategy employed, and the results obtained from testing and evaluation.

4.2 Mapping Logical Design onto Physical Platform

The logical architecture described in Chapter Three was implemented using the following technology stack:

Layer	Technology / Tool	Justification
Front-end (Web)	React.js, HTML5, CSS3, Bootstrap/Tailwind CSS	Component-based structure supports a responsive, maintainable customer and admin interface.
Front-end (Mobile)	React Native / Flutter	Enables a single codebase to target both Android and iOS platforms, reducing development effort.
Back-end / API	Node.js with Express.js (RESTful API)	Lightweight, widely supported, and well suited to handling asynchronous I/O such as payment callbacks and notifications.
Database	MySQL (relational database)	Well suited to the structured, relational nature of customer, order, and payment data; supports foreign-key constraints and transactions.
Payment Integration	Mobile Money API (e.g., MTN MoMo API) and card payment gateway (e.g., Paystack/Stripe sandbox)	Provides secure, PCI-DSS-compliant handling of payment details relevant to the local and international context.
Notifications	SMS gateway API and email service (SMTP)	Enables automated status notifications to customers without requiring the app to be open.
Hosting	Cloud hosting platform (e.g., Render/Heroku/AWS) with version control via Git/GitHub	Provides scalable, always-on hosting with straightforward deployment and rollback capability.

4.3 System Modules Implementation

4.3.1 Customer Registration and Authentication Module

This module was implemented to allow a new customer to create an account by providing their full name, phone number, email address, and a password, which is hashed using bcrypt before being stored in the database. On subsequent visits, the customer logs in using their phone number/email and password, and the server issues a signed JSON Web Token (JWT) used to authenticate subsequent API requests. Basic input validation (e.g., valid phone number format, minimum password length) is enforced both on the client and the server.

4.3.2 Order Processing Module

The order processing module allows an authenticated customer to browse the list of services retrieved from the Service table, select a service, specify item quantity/weight, and choose a pickup date, time, and address. On submission, the module calculates the total cost based on the service price and quantity, creates a new record in the Order table with a status of “Pending,” and returns an order summary to the customer for confirmation.

4.3.3 Payment Management Module

Once an order is confirmed, the payment module presents the customer with a choice of payment method — mobile money or card. The selected payment request is forwarded to the relevant third-party payment gateway API. Upon receiving a successful payment callback/webhook from the gateway, the system records the transaction in the Payment table, updates the order status to “Confirmed,” and generates a digital receipt that is emailed to the customer. Failed transactions are logged with a status of “Failed,” and the customer is prompted to retry payment.

4.3.4 Order Tracking Module

This module allows administrative staff and riders to update the status of an order as it progresses through defined stages (Picked Up, Washing, Drying, Ironing, Ready, Delivered). Each update creates a new record in the OrderTracking table, capturing the responsible staff/rider ID and a timestamp. Customers view these updates in real time on the Order Tracking screen, and an automated notification (SMS/email/push) is triggered on every status change.

4.3.5 Administrative Dashboard Module

The administrative dashboard provides laundry management staff with a consolidated view of all orders (filterable by status and date), tools to assign pickup/delivery tasks to riders, management of customer and staff/rider accounts, and a reporting screen that aggregates order volume and revenue data over selectable date ranges using summary charts.

4.4 System Modules Integration

The individual modules described above were integrated through the central RESTful API, which serves as the single point of communication between all front-end clients (customer web/mobile app, admin dashboard, rider interface) and the back-end business logic and database. Integration was carried out incrementally, in line with the process model described in Chapter Three: the registration/authentication module was integrated and tested first, followed by order processing, then payment management, and finally order tracking and reporting, with each newly integrated module tested against the previously integrated modules to catch regression issues early.

4.4.1 Testing

A structured testing strategy was followed to verify that the integrated system met its functional and non-functional requirements.

4.4.2 Testing Plan

The testing plan covered four levels of testing, executed in the following order:

1. Unit Testing – testing individual functions and components (e.g., password hashing, total cost calculation) in isolation.
2. Integration Testing – testing the interaction between modules (e.g., order creation followed by payment processing and status update).
3. System/Validation Testing – testing the complete, integrated system against the functional requirements specified in Chapter Three.
4. User Acceptance Testing (UAT) – having representative users (a laundry attendant and prospective customers) use the system and provide feedback.

4.4.3 Verification Testing

Verification testing ensured that each module was built correctly according to its design specification (“building the product right”). This was carried out through unit tests on individual functions (e.g., verifying that `calculateTotal()` returns the correct amount for a given service and quantity) and code review/walkthroughs of each module before integration.

4.4.4 Validation Testing

Validation testing confirmed that the system, as a whole, satisfied the stated functional requirements (“building the right product”). Test cases were derived directly from the functional requirements listed in Section 3.4, and each was executed against the running system.

Test ID	Test Case	Expected Result	Actual Result
TC-01	Register a new customer account with valid details	Account created; confirmation shown	Passed
TC-02	Attempt registration with an already-registered phone number	System rejects duplicate registration with an error message	Passed
TC-03	Place an order by selecting a service and scheduling pickup	Order created with status 'Pending' and correct total cost	Passed
TC-04	Complete payment for a pending order via mobile money (sandbox)	Payment recorded; order status updated to 'Confirmed'; receipt generated	Passed
TC-05	Simulate a failed payment transaction	Order status remains 'Pending'; customer prompted to retry	Passed
TC-06	Admin assigns a rider to a confirmed order	Order shows assigned rider; rider receives task on their interface	Passed

Test ID	Test Case	Expected Result	Actual Result
TC-07	Rider updates order status to 'Picked Up'	Customer's tracking screen reflects updated status in real time	Passed
TC-08	Generate a revenue report for a selected date range	System displays accurate total revenue and order count for the range	Passed
TC-09	Attempt to access the admin dashboard as a customer account	Access denied due to role-based access control	Passed

4.4.5 System Security Testing

Security testing focused on verifying the safeguards described in Section 3.8. Tests included attempting to log in with incorrect credentials to confirm appropriate lockout/error behaviour, attempting basic SQL injection strings in input fields to confirm that parameterised queries prevented injection, verifying that passwords were not retrievable in plain text from the database, confirming that all API endpoints required a valid authentication token except for public endpoints (e.g., service listing), and confirming that role-based access control correctly restricted staff-only and admin-only endpoints from customer accounts. All security tests carried out passed, with no critical vulnerabilities identified within the scope of testing performed.

4.4.6 Recommendations Made by Testers

- Add a password-strength indicator during registration to encourage stronger customer passwords.
- Provide an option for customers to cancel a pending order before it is picked up.
- Add a confirmation prompt before a rider marks an order as 'Delivered' to reduce accidental status updates.
- Improve error messages for failed mobile money payments to clarify the specific reason for failure (e.g., insufficient funds vs. network timeout).

4.4.7 Responses to Recommendations from Testing

- A password-strength indicator was implemented on the registration form, requiring a minimum of one uppercase letter, one number, and eight characters.
- An order cancellation option was added, restricted to orders still in the 'Pending' status.
- A confirmation dialog was added to the rider interface before finalising a 'Delivered' status update.
- Payment failure messages were updated to surface the specific error returned by the payment gateway sandbox where available, with a generic fallback message otherwise.

4.5 Results

Following implementation and testing, the Laundry Management System successfully achieved the objectives set out in Chapter One. All nine core test cases derived from the functional requirements passed, confirming that customer registration, order processing, payment management, and order tracking function correctly and consistently. User acceptance testing feedback from the participating laundry attendant and prospective customers indicated that the system was considerably easier to use

than the existing manual process, with particular appreciation for the ability to track order status remotely and receive automated notifications, eliminating the need for customers to call or visit the shop to check on their laundry. The average time to process and confirm a new order through the system was measured at under one minute, compared to several minutes typically required to manually record an order and calculate its cost. These results confirm that the system meets its stated aim of automating and improving the efficiency of the core laundry business processes.

CHAPTER FIVE

FINDINGS, CONCLUSIONS AND RECOMMENDATIONS

5.1 Introduction

This chapter presents a summary of the key findings from the design, implementation, and testing of the Laundry Management System, draws conclusions regarding the extent to which the project objectives were achieved, discusses the challenges encountered during the project and the lessons learnt, and offers recommendations for future enhancement of the system.

5.2 Findings

- Automating customer registration, order processing, payment management, and order tracking substantially reduces the time and effort required to manage a laundry business compared to manual, paper-based methods.
- Real-time order status tracking was the single most appreciated feature during user acceptance testing, as it removes the need for customers to physically visit or call the laundry shop for updates.
- Integrating a recognised mobile money and card payment gateway significantly improves payment transparency and allows for automatic reconciliation of revenue against orders.
- A role-based access control model is essential in a system with multiple distinct user types (customer, staff, rider, admin) to ensure that each user only interacts with functionality relevant to their responsibilities.
- Adopting an incremental development process allowed issues in early modules (e.g., authentication) to be identified and resolved before they could compound in later modules (e.g., payment), reducing overall integration risk.

5.3 Conclusions

The project successfully achieved its stated aim of designing and implementing a web and mobile-based Laundry Management System that automates customer registration, order processing, payment management, and order tracking. Each of the specific objectives outlined in Chapter One was met: related systems were reviewed and their strengths incorporated into the conceptual design; functional and non-functional requirements were elicited and documented; a complete system architecture, set of UML diagrams, and database schema were designed; and the core modules were implemented, integrated, and tested with all derived test cases passing successfully. The system demonstrates that even a small or medium-sized laundry business can benefit from a digitised, automated workflow without the cost and complexity associated with large international on-demand platforms, provided the system is scoped appropriately to the business's actual needs and resources.

5.4 Challenges

- Limited access to a live, production-grade payment gateway account meant that payment integration had to be tested primarily using sandbox/test credentials, which may not fully capture all edge cases encountered in live transactions.

- Coordinating requirements gathering with a real laundry business owner and attendant within the constrained academic timeline required careful scheduling and, at times, resulted in delayed feedback.
- Balancing the desire for real-time GPS-based rider tracking (as seen in some reviewed systems) against the additional infrastructure and cost such a feature would require led to the adoption of a simplified, status-based tracking approach instead.
- Ensuring consistent behaviour of the notification module across multiple channels (SMS, email, and push notifications) required additional debugging time due to differing API response formats across providers.

5.5 Lessons Learnt

- Early and continuous engagement with prospective end-users (the laundry attendant and customers) during requirements gathering and user acceptance testing was invaluable in surfacing practical concerns that might otherwise have been overlooked in a purely technical design process.
- Adopting an incremental process model, rather than attempting to build and integrate the entire system at once, made it considerably easier to isolate and fix defects as they arose.
- Designing the database schema carefully at the outset, including appropriate normalisation, prevented significant rework later in the implementation phase.
- Security cannot be an afterthought: incorporating password hashing, role-based access control, and input validation from the earliest implementation stages was far more effective than attempting to retrofit these measures later.

5.6 Recommendations for Future Works

1. Integrate live GPS-based tracking for riders to provide customers with a more granular, real-time view of their delivery's location, similar to the approach used by Laundryheap.
2. Introduce a loyalty/rewards programme to encourage repeat customers, drawing on the subscription and preference-management concepts observed in Cleanly.
3. Extend the system to support multiple laundry branches or a multi-tenant model, allowing several independent laundry businesses to operate on the same platform, similar to the marketplace model used by Laundrapp.
4. Develop a dedicated analytics dashboard with predictive insights (e.g., forecasting peak order periods) to support better staffing and inventory decisions.
5. Conduct a full pilot deployment with a live, production payment gateway account and a real laundry business over an extended period to gather more comprehensive real-world performance and usage data.
6. Implement offline-first capability on the mobile application so that riders and staff in areas with unreliable internet connectivity can continue recording status updates that synchronise once connectivity is restored.